Chapitre 3:
Les branches avec Git

Plan



1. Qu’est ce qu’une branche
2. Créer une nouvelle branche
3. Basculer entre les branches
4. Fusion et rebasage
5. TPs 3 et 4

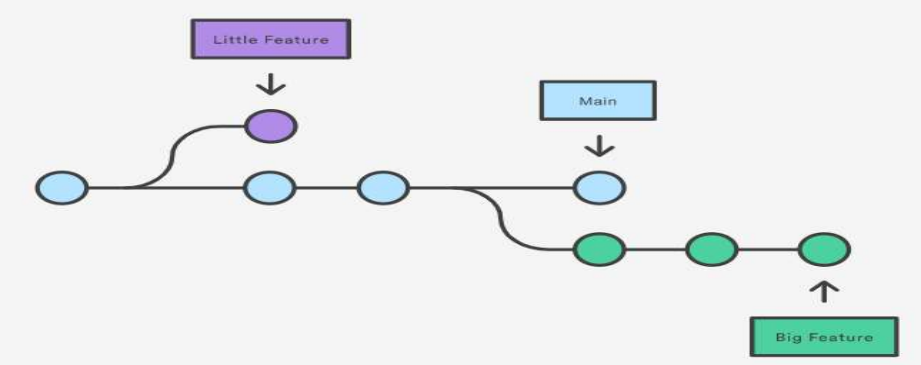
Qu’est ce qu’une branche

Définition

- Dans Git, créer une branche consiste tout simplement **à créer un nouveau pointeur vers un commit spécifique**, contrairement à d'autres systèmes de contrôle de version qui créent une copie physique du répertoire de travail, ce qui rend la création de branches lourde et contraignante.

- La branche par défaut dans Git s'appelle **"master",** qui est traitée de la même manière que les autres branches et qui est simplement renommable.

- En résumé, une branche est un pointeur vers un commit spécifique et créer une nouvelle branche dans Git crée simplement un nouveau pointeur plutôt que de recopier l'intégralité du répertoire de travail.



Le développement de fonctionnalités dans des branches vous permet de travailler sur deux fonctionnalités en parallèle. De plus, vous évitez de polluer la branche principale (main) avec du code douteux.

Créer une nouvelle branche

Créer une branche

- Pour créer une nouvelle branche, on utilise la commande **git branch** nom-de-la-branche. Cette syntaxe va créer un nouveau pointeur vers le dernier commit effectué (le commit courant). A ce stade, **vous allez donc avoir deux branches (deux pointeurs) vers le dernier commit : la branche master et la branche tout juste créée.**

```
TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/Desktop/tpCoursBranches (master)
$ git log --oneline
82c6ade (HEAD -> master) 1ère maj
068c083 commit 3
64fb884 commit 2
e746a25 commit 1

TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/Desktop/tpCoursBranches (master)
$ git branch test

TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/Desktop/tpCoursBranches (master)
$ git status
On branch master
nothing to commit, working tree clean
```




```
TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/Desktop/tpCoursBranches (master)
$ git branch
* master
  test
```

- La commande **git branch** permet de lister toutes les branches de votre répertoire = **git branch --list**
- Ici, on pointe sur la branche **master.**

Créer une nouvelle branche

- Pour déterminer quel pointeur vous utilisez, c'est-à-dire sur quelle branche vous vous trouvez, Git utilise un autre pointeur spécial appelé HEAD. HEAD pointe sur la branche master par défaut. Notez que la commande git branch permet de créer une nouvelle branche mais ne déplace pas HEAD.

- Nous allons donc devoir déplacer explicitement HEAD pour indiquer à Git qu'on souhaite basculer sur une autre branche. On utilise pour cela la commande git checkout suivi du nom de la branche sur laquelle on souhaite basculer.

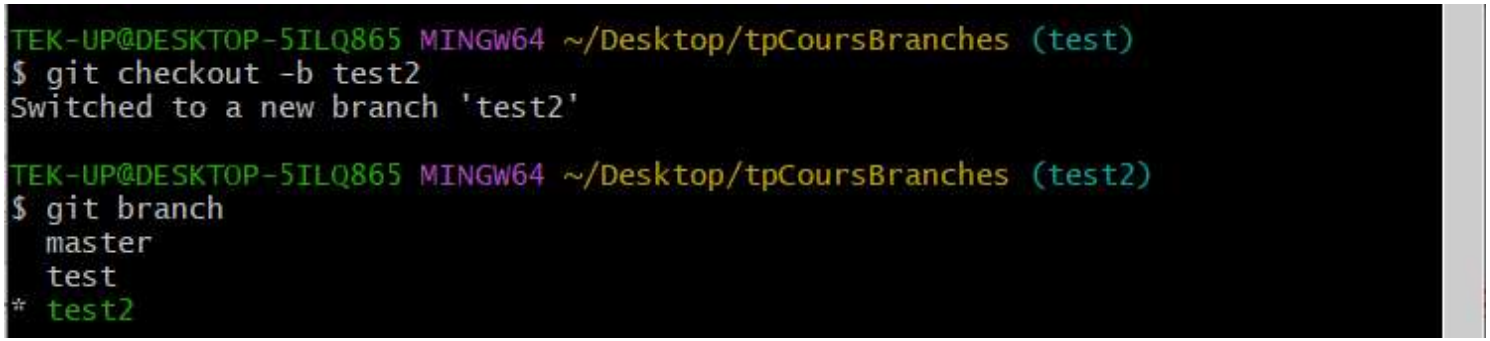
```
TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/Desktop/tpCoursBranches (master)
$ git checkout test
Switched to branch 'test'

TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/Desktop/tpCoursBranches (test)
$ git branch
  master
* test
```



Créer une nouvelle branche

- Note : On peut également utiliser **git checkout –b** nom-de-la-branche pour créer une branche et basculer immédiatement dessus. Cela est l'équivalent d'utiliser **git branch** nom-de-la-branche puis **git checkout** nom-de-la-branche.



Créer une nouvelle branche

```
TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/Desktop/tpCoursBranches (test2)
$ git checkout test
Switched to branch 'test'

TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/Desktop/tpCoursBranches (test)
$ git branch
  master
* test
  test2
```

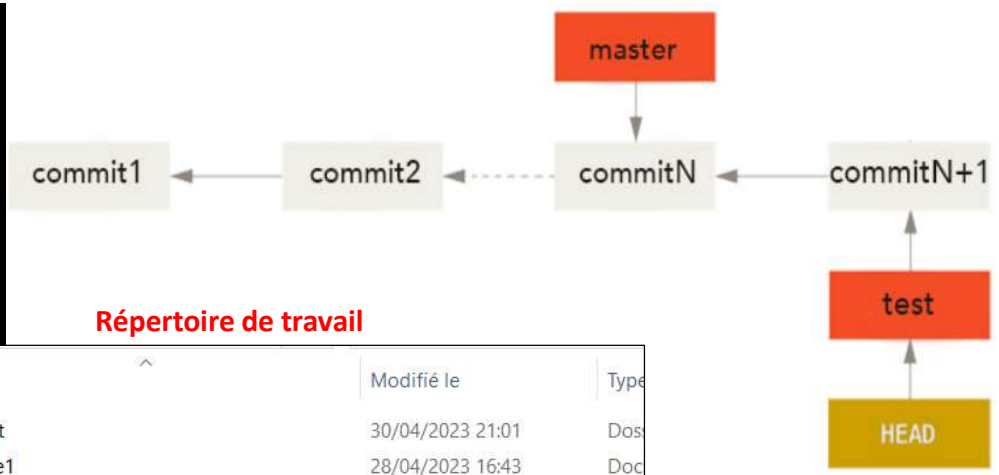
```
TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/Desktop/tpCoursBranches (test)
$ touch file4.txt

TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/Desktop/tpCoursBranches (test)
$ git add file4.txt

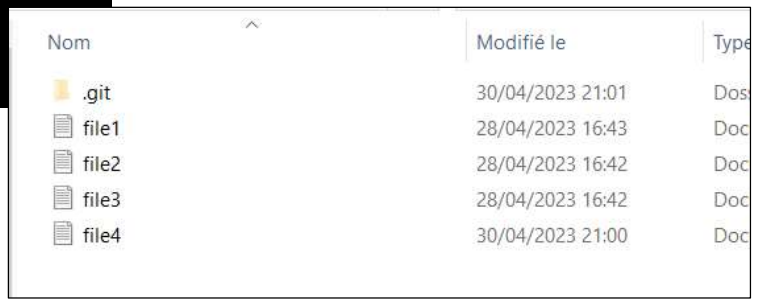
TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/Desktop/tpCoursBranches (test)
$ git commit -m "nouveau fichier file4 ajouté"
[test 5003d1d] nouveau fichier file4 ajouté
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 file4.txt

TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/Desktop/tpCoursBranches (test)
$ git status
On branch test
nothing to commit, working tree clean
```

HEAD pointe maintenant vers notre branche test. Si on effectue un nouveau commit, la branche test va avancer automatiquement tandis que master va toujours pointer sur le commit précédent. C'est en effet la branche sur laquelle on se situe lors d'un commit qui va pointer sur ce commit.



Répertoire de travail



Créer une nouvelle branche

Créer une branche distante

- Les exemples précédents ont tous démontré des opérations de branches locales.

- La commande **git branch** fonctionne également sur les branches distantes.

- Pour opérer sur des branches distantes, **un dépôt distant doit être configuré et ajouté à la configuration du dépôt local**.
- **$ git remote add origin <url du dépôt>**

  # Ajouter un dépôt distant à la configuration du dépôt local

- **$ git push <origin> branche1** # pushes la branche branche1 vers origin
- Cette commande poussera une copie de la branche locale branche1 vers le dépôt distant ＜remote＞.

Quelques commandes utiles

- $ git branch --list //Répertoriez toutes les branches de dépôt.
- $ git branch <branch> // Créer une nouvelle branche appelée <branch>.
- $ git branch -d <branch_name> // Supprimer la branche spécifiée, Git vous empêche de supprimer la branche si elle contient des modifications non fusionnées
- $ git branch -D <branch_name> // Forcer la suppression de la branche spécifiée, même si elle contient des modifications non fusionnées.
- $ git branch -m <branch> // Renommer la branche actuelle en <branch>.
- $ git branch –m old_name new_name // Renommer la branche.
- $ git branch –a // Répertorier toutes les branches distantes (Pour lister toutes les branches).
- $ git checkout branch_name // Pour passer à une branche spécifique
- $ git push origin --delete name_branch // Pour supprimer une branche distante.
- Ou bien :
- $ git push origin : name_branch // Pour supprimer une branche distante.
- Cela enverra un signal de suppression au dépôt origin distant qui déclenchera une suppression de la branche distante.
- $ git checkout -b "branch_name" // créer et switchez dans une branche à la fois

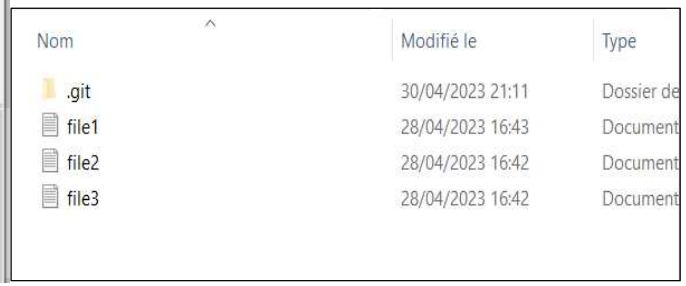
Basculer entre les branches

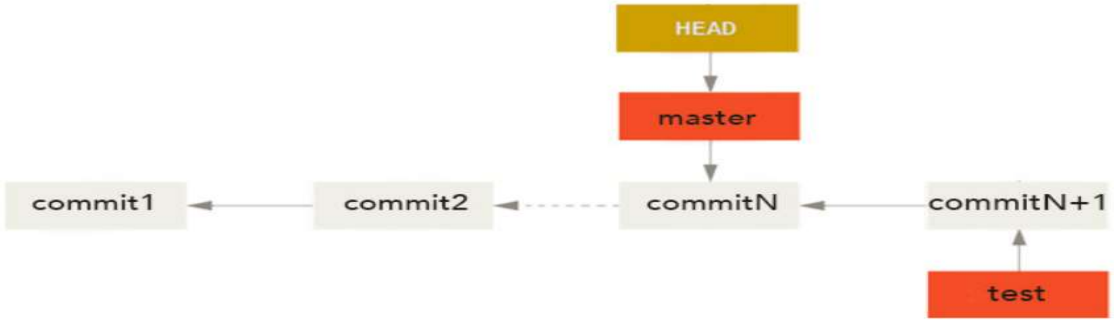
- On peut revenir sur notre branche master en tapant à nouveau une commande git checkout master. Cela replace le pointeur HEAD sur la branche master et restaure le répertoire de travail dans l'état de l'instantané pointé par le commit sur lequel pointe master.

```
TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/Desktop/tpCoursBranches (test)
$ git checkout master
Switched to branch 'master'

TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/Desktop/tpCoursBranches (master)
$ git status
On branch master
nothing to commit, working tree clean
```

Répertoire de travail





Basculer entre les branches

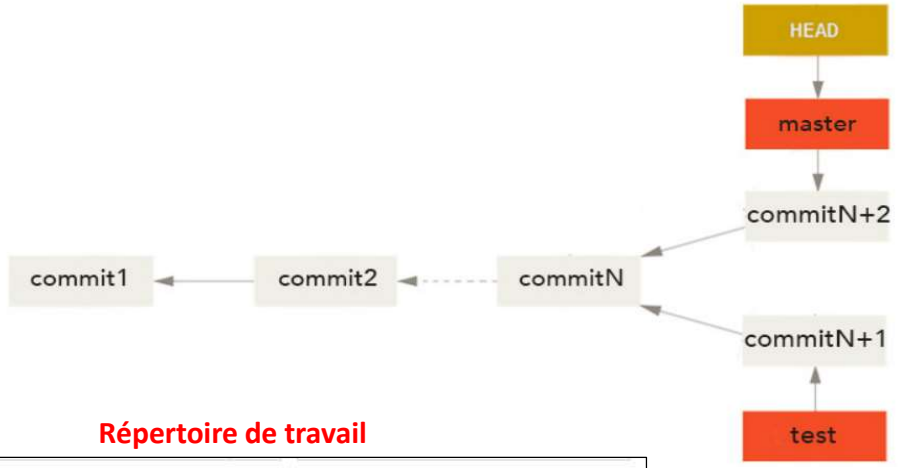
- On Si on modifie alors le répertoire de travail et qu'on effectue un nouveau commit, les deux branches master et test vont diverger. On va donc avoir deux branches pointant vers des instantanés qui ont enregistré le projet dans deux états différents.

```
TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/Desktop/tpCoursBranches (master)
$ touch file5.txt

TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/Desktop/tpCoursBranches (master)
$ git add file5.txt

TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/Desktop/tpCoursBranches (master)
$ git commit -m "nouveau fichier file5 ajouté"
[master b38db4b] nouveau fichier file5 ajouté
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 file5.txt

TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/Desktop/tpCoursBranches (master)
$ git log --oneline
b38db4b (HEAD -> master) nouveau fichier file5 ajouté
82c6ade (test2) 1ère maj
068c083 commit 3
64fb884 commit 2
e746a25 commit 1
```



Répertoire de travail

Nom	Modifié le	Type
.git	30/04/2023 21:23	Dossier de
file1	28/04/2023 16:43	Document
file2	28/04/2023 16:42	Document
file3	28/04/2023 16:42	Document
file5	30/04/2023 21:22	Document

Fusion et rebasage

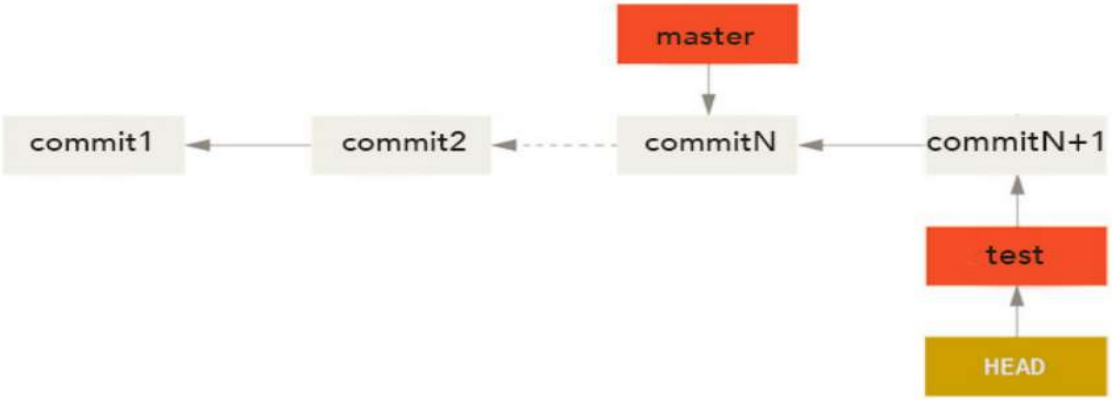
Récap

- Dans Git, Les branches permettent de travailler sur de nouvelles fonctionnalités sans impacter la branche principale.
- Les fonctionnalités sont développées sur des branches connexes et testées avant d'être réintégrées dans la branche principale.
- Pour réintégrer les fonctionnalités, il faut rapatrier le contenu des branches dans la branche principale.
- Git propose deux méthodes pour cela : la fusion des branches ou le rebasage.

Fusionner des branches

Exemple 1 :

- Commençons avec un cas plus simple et imaginons que notre projet soit dans cet état :



Ici, on a une branche test qui pointe sur un commit commitN+1 et une branche master qui pointe sur un commit commitN. commitN est l'ancêtre direct de commitN+1 et il n'y a donc pas de problème de divergence.

- Pour fusionner nos deux branches, on va se placer sur master avec une commande git checkout puis taper une commande git merge avec le nom de la branche qu'on souhaite fusionner avec master.

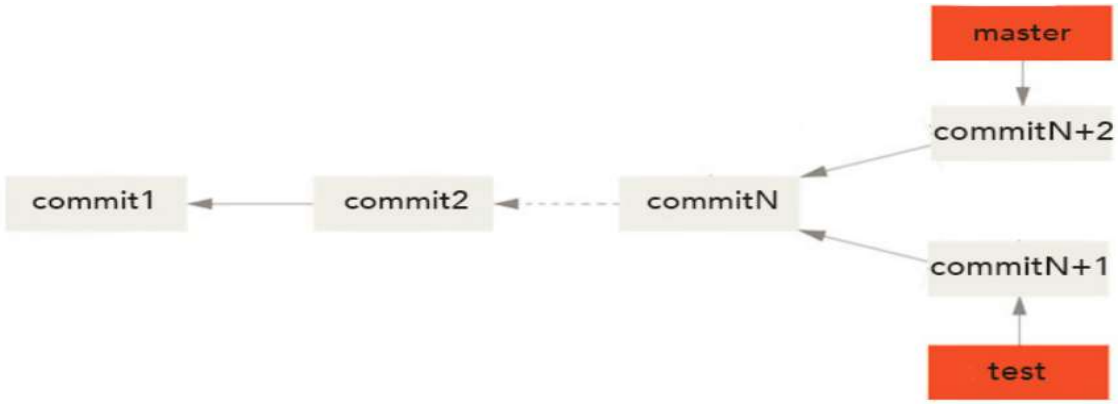
- Dans ce cas, "fusionner" nos deux branches revient finalement à faire avancer master au niveau du commit pointé par test. C'est exactement ce que fait Git et c'est ce qu'on appelle un "fast forward" (avance rapide).

- Il ne nous reste alors plus qu'à effacer notre branche test. On peut faire cela en utilisant la commande git branch -d.

Fusionner des branches

Exemple 2 :

Reprenons maintenant la situation précédente avec deux branches dont les historiques divergent. On peut représenter cette situation comme cela :



- Pour fusionner deux branches ici on va à nouveau se placer dans la branche dans laquelle on souhaite fusionner puis effectuer un git merge.

- Dans cet exemple, Git réalise une fusion en utilisant 3 sources : le dernier commit commun aux deux branches et le dernier commit de chaque branche.

Fusionner des branches

Exemple 2 :

- Git crée automatiquement un nouvel instantané dont le contenu est le résultat de la fusion ainsi qu'un commit qui pointe sur cet instantané.

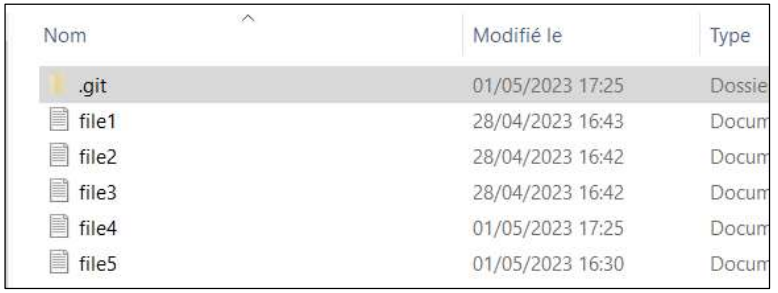
- Ce commit s'appelle **un commit de fusion** et est spécial puisqu'il possède plusieurs parents.

```
TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/Desktop/tpCoursBranches (master)
$ git branch
* master
  test

TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/Desktop/tpCoursBranches (master)
$ git merge test
Merge made by the 'ort' strategy.
 file4.txt | 0
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 file4.txt

TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/Desktop/tpCoursBranches (master)
$ git log --oneline
618eeac (HEAD -> master) Merge branch 'test'
b38db4b nouveau fichier file5 ajouté
5003d1d (test) nouveau fichier file4 ajouté
82c6ade 1ère maj
068c083 commit 3
64fb884 commit 2
e746a25 commit 1
```

Répertoire de travail



Fusionner des branches

- La commande **"git merge --squash"** fusionne une branche dans une autre branche sans créer de commit de fusion.
- Toutes les modifications de la branche fusionnée sont rassemblées en un seul commit sur la branche de destination.
- Vous pouvez inclure un message de commit personnalisé pour le commit de fusion en utilisant la commande "git commit".
- La commande "git merge --squash" est utile lorsque vous ne voulez pas inclure l'historique complet de la branche source dans la branche de destination.
- La commande "git merge --squash" ne supprime pas la branche source après la fusion, vous devez la supprimer manuellement en utilisant la commande "git branch -d".

Fusionner des branches

- Notez que dans le cas d'une fusion à trois sources, il se peut qu'il y ait des conflits. Cela va être notamment le cas si une même partie d'un fichier a été modifiée de différentes manières dans les différentes branches. Dans ce cas, lors de la fusion, Git nous alertera du conflit et nous demandera de le résoudre avant de terminer la fusion des branches.

- Exemple : Imaginons que nos deux branches possèdent un fichier LISEZMOI.txt et que les deux fichiers LISEZMOI.txt possèdent des textes différents. Git va automatiquement "fusionner" les contenus des deux fichiers en un seul qui va en fait contenir les textes des deux fichiers de base à la suite l'un de l'autre avec des indicateurs de séparation.

- On peut alors ouvrir le fichier à la main et choisir ce qu'on conserve (en supprimant les parties qui ne nous intéressent pas par exemple). Dès qu'on a terminé l'édition, on va taper une commande git add pour marquer le conflit comme résolu. On n'aura alors plus qu'à effectuer un git commit pour terminer le commit de fusion.

Rebaser des branches

- Git nous fournit deux moyens de rapatrier le travail effectué sur une branche vers une autre : on peut fusionner ou **rebaser.**

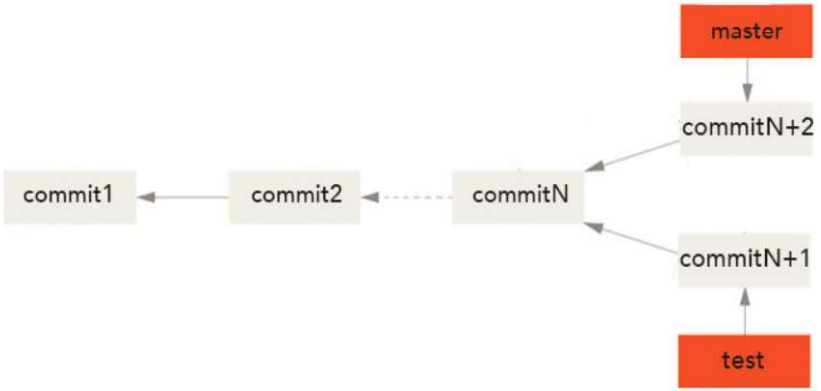
- Reprenons notre exemple précédent avec nos deux branches divergentes.

```
TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/Desktop/tpCoursBranches (master)
$ git log --oneline
618eeac (HEAD -> master) Merge branch 'test'
b38db4b nouveau fichier file5 ajouté
5003d1d (test) nouveau fichier file4 ajouté
82c6ade 1ère maj
068c083 commit 3
64fb884 commit 2
e746a25 commit 1

TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/Desktop/tpCoursBranches (master)
$ git branch
* master
  test

TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/Desktop/tpCoursBranches (master)
$ git reset --hard b38db4b
HEAD is now at b38db4b nouveau fichier file5 ajouté

TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/Desktop/tpCoursBranches (master)
$ git log --oneline
b38db4b (HEAD -> master) nouveau fichier file5 ajouté
82c6ade 1ère maj
068c083 commit 3
64fb884 commit 2
e746a25 commit 1
```



Ici, **pour revenir à cet état,** on a exécuté la commande **$git reset --hard b38db4b** (dernier commit de la branche master avant l'étape de fusion == on vient d'annuler l'opération de fusion)

Rebaser des branches

- Plutôt que d'effectuer une fusion à trois sources, on va pouvoir rebaser les modifications validées dans commitN+1 dans notre branche master. On utilise la commande git rebase pour récupérer les modifications validées sur une branche et les rejouer sur une autre.

- Dans ce cas, Git part à nouveau **du dernier commit commun aux deux branches** (l'ancêtre commun le plus récent) puis récupère les modifications effectuées sur la branche qu'on souhaite rapatrier et les applique sur la branche vers laquelle on souhaite rebaser notre travail **dans l'ordre dans lequel elles ont été introduites.**

```
TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/Desktop/tpCoursBranches (master)
$ git log --oneline
b38db4b (HEAD -> master) nouveau fichier file5 ajouté
82c6ade 1ère maj
068c083 commit 3
64fb884 commit 2
e746a25 commit 1

TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/Desktop/tpCoursBranches (master)
$ git rebase test
Successfully rebased and updated refs/heads/master.

TEK-UP@DESKTOP-5ILQ865 MINGW64 ~/Desktop/tpCoursBranches (master)
$ git log --oneline
98aabd3 (HEAD -> master) nouveau fichier file5 ajouté
5003d1d (test) nouveau fichier file4 ajouté
82c6ade 1ère maj
068c083 commit 3
64fb884 commit 2
e746a25 commit 1
```

Rebaser rejoue les modifications d'une ligne de commits sur une autre dans l'ordre d'apparition.

Comparaison

- La principale différence entre "git merge" et "git rebase" est que "git merge" fusionne deux branches en créant un nouveau commit de fusion, tandis que "git rebase" réapplique l'historique de commit de la branche source à la branche de destination en réécrivant l'historique des commits.

- "git merge" conserve l'historique de commit original de chaque branche, tandis que "git rebase" réécrit l'historique des commits de la branche source.

- "git merge" est utilisé pour fusionner des branches avec des modifications importantes et distinctes, tandis que "git rebase" est utilisé pour intégrer des modifications mineures dans une branche principale, pour réorganiser l'historique de commit ou pour maintenir un historique de commit linéaire.

- "git merge" peut introduire des conflits de fusion qui doivent être résolus manuellement, tandis que "git rebase" peut introduire des conflits de réécriture d'historique qui peuvent également nécessiter une résolution manuelle.

git merge vs git rebase

- Le choix entre "git merge" et "git rebase" dépend des besoins du projet et des préférences de l'équipe de développement. Voici quelques critères à considérer :
  - Si vous voulez conserver l'historique de commit original de chaque branche, utilisez "git merge".
  - Si vous voulez maintenir un historique de commit linéaire et plus facile à suivre, utilisez "git rebase".
  - Si vous voulez fusionner des branches avec des modifications importantes et distinctes, utilisez "git merge".
  - Si vous voulez intégrer des modifications mineures dans une branche principale, utilisez "git rebase".
  - Si vous voulez réorganiser l'historique de commit, utilisez "git rebase".
  - Si vous voulez éviter les conflits de fusion, utilisez "git rebase".
  - Si vous voulez éviter de réécrire l'historique de commit, utilisez "git merge".
- En fin de compte, il n'y a pas de réponse unique pour le choix entre "git merge" et "git rebase". Le choix dépend des besoins du projet, de la préférence de l'équipe de développement et de la façon dont vous voulez structurer l'historique de commit.

TP3 : Notion des Branches

TP4 : git merge